

Taller — Documentación de requerimientos del proyecto

Propósito: producir un paquete de documentación de requerimientos claro, trazable y verificable para una app móvil, siguiendo la **plantilla SRS IEEE 830** y complementando con **historias de usuario**, **casos de uso** (con especificación y diagrama) y **matriz de trazabilidad**.

Escenario guía (ejemplo): *CampusGo*, una app móvil para estudiantes que permite registro/login, visualización de eventos del campus, y gestión de favoritos offline.

1) Entregable principal — SRS (IEEE 830)

Qué es: un **Software Requirements Specification** (SRS) que describe el producto con precisión suficiente para diseño, pruebas y validación. A continuación se presenta la plantilla con **instrucciones** y un **ejemplo resumido** por cada apartado (basado en *CampusGo*).

1.1 Introducción

- **1.1.1 Propósito** — ¿Para quién es el SRS y qué objetivo tiene?

Ejemplo: Este SRS sirve al equipo de desarrollo y QA para definir el alcance y criterios de aceptación de CampusGo.

- **1.1.2 Alcance** — Qué hace (y qué no) el sistema.

Ejemplo: CampusGo permite registro/login, ver eventos, marcar favoritos; no incluye compra de entradas ni pagos.

- **1.1.3 Definiciones, acrónimos y abreviaturas** — Glosario.

Ejemplo: RF = Requisito Funcional; RNF = Requisito No Funcional.

- **1.1.4 Referencias** — Normas, documentos y enlaces.
- **1.1.5 Visión general del documento** — Cómo está organizado.

1.2 Descripción general

- **1.2.1 Perspectiva del producto** — Sistema en su contexto (móvil, backend, APIs).

Ejemplo: App móvil nativa/híbrida que consume API REST; backend propio con DB.

- **1.2.2 Funciones del producto** — Lista de funciones principales (alto nivel).
 - Registro y autenticación.
 - Explorar eventos.
 - Favoritos offline.
- **1.2.3 Características de los usuarios** — Perfiles, habilidades.

Ejemplo: Estudiantes con alfabetización digital básica; personal administrativo.

- **1.2.4 Restricciones** — Técnicas/legales/negocio (tiempo, tienda, privacidad).
- **1.2.5 Supuestos y dependencias** — Ej.: conectividad, API de eventos, políticas de TI.

1.3 Requerimientos específicos

Divídelos por categorías. Sé **medible**, **no ambiguo** y **verificable**.

- **1.3.1 Interfaces externas**
 - UI, API, hardware, comunicaciones (endpoints, formatos).

Ej.: GET /api/events?limit=20 – Respuesta: 200 OK [{id, title, starts_at, ...}]

- **1.3.2 Funciones del sistema (RF)**

ID	Descripción	Prioridad	Criterio de aceptación
----	-------------	-----------	------------------------

RF-01	El sistema permitirá registrar usuarios con correo y contraseña.	Alta	Dados email válido y contraseña ≥ 8 , al enviar, se crea la cuenta y se envía confirmación.
RF-02	El usuario podrá ver un listado de eventos con búsqueda por texto.	Media	Al buscar “música”, se listan eventos cuyo título/descripción coincida.
RF-03	El usuario podrá marcar y ver eventos como “favoritos” offline.	Alta	En modo avión, el listado de favoritos sigue visible.

- **1.3.3 Rendimiento (RNF-Performance)** — tiempos, concurrencia, límites.

Ejemplo: p95 de arranque ≤ 2.5 s en gama media; listado de eventos en ≤ 800 ms.

- **1.3.4 Lógica de datos / base de datos** — entidades y relaciones.

Tabla EVENTS: id (PK), title, starts_at, location, updated_at
Tabla FAVORITES: user_id (FK), event_id (FK), created_at

- **1.3.5 Restricciones de diseño** — plataformas, SDKs, guías UI.
- **1.3.6 Atributos del sistema (RNF)** — seguridad, disponibilidad, mantenibilidad, portabilidad, accesibilidad.
 - **Seguridad:** credenciales cifradas, TLS 1.2+, bloqueo tras 5 intentos fallidos.
 - **Accesibilidad:** contraste AA, targets táctiles $\geq 44 \times 44$ px, soporte lector de pantalla.
- **1.3.7 Requisitos de internacionalización/localización** (si aplica).
- **1.3.8 Requisitos legales y de privacidad** (si aplica).

1.4 Apéndices (opcional)

- Mockups iniciales, DER simplificado, políticas de datos, etc.

Importante — El SRS debe ser la única “verdad” del alcance: si una funcionalidad no está en el SRS, no se implementa en esta etapa.

2) Historias de usuario (documentadas y trazables)

Complementan al SRS con enfoque de valor para el usuario. Cada historia debe tener **prioridad**, **criterios de aceptación** y **trazas** a RF/RNF.

2.1 Plantilla recomendada (tabla)

ID	Como (rol)	Quiero (objetivo)	Para (beneficio)	Prioridad	RF/RNF relacionados
HU-01	Estudiante	registrarme con correo	acceder a las funciones de la app	Alta	API /email
HU-02	Estudiante	ver eventos por fecha	planificar mi asistencia	Media	Catálogo eventos
HU-03	Estudiante	marcar favoritos offline	consultarlos sin conexión	Alta	Storage local

2.2 Criterios de aceptación (Gherkin)

Funcionalidad: Favoritos offline

Escenario: Acceso a favoritos sin conexión

Dado que marqué 2 eventos como favoritos con conectividad

Y que activo el modo avión

Cuando accedo a la sección "Favoritos"

Entonces debo ver esos 2 eventos listados

Y no debe mostrarse error de red

Buenas prácticas: historias *INVEST* (Independientes, Negociables, con Valor, Estimables, Pequeñas, Testeables).

3) Casos de uso (explicación + especificación + diagrama)

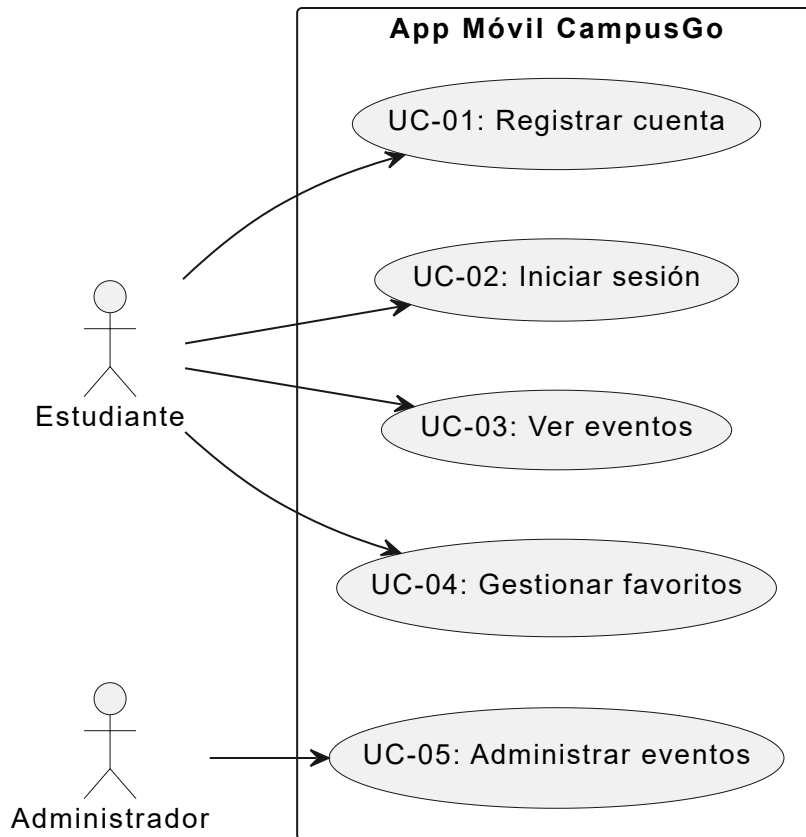
3.1 ¿Qué es un caso de uso?

Una descripción de **interacciones** entre actores y el sistema para conseguir un objetivo de negocio. Útil para detallar **flujos** y **excepciones**.

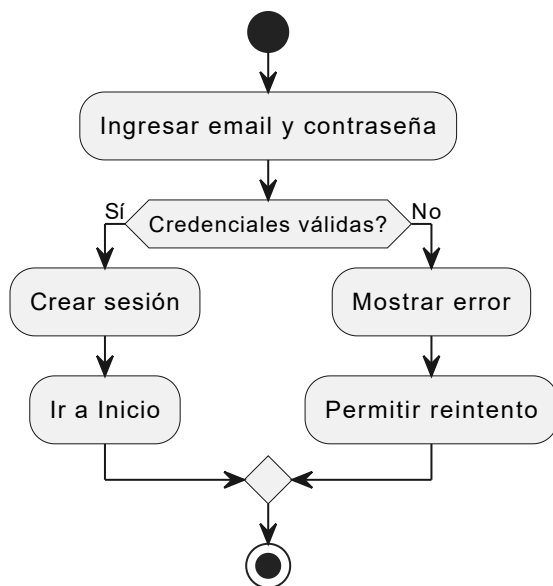
3.2 Plantilla de especificación de caso de uso

Campo	Descripción
ID	UC-02
Nombre	Iniciar sesión
Actor primario	Estudiante
Interesados	Estudiante (acceso), Soporte (reducción incidencias)
Precondiciones	Usuario registrado; app instalada
Postcondiciones (éxito)	Sesión activa y acceso a pantalla Inicio
Postcondiciones (fallo)	Mensaje de error y posibilidad de reintentar
Flujo principal	1) Usuario ingresa correo y contraseña. 2) Sistema valida credenciales. 3) Sistema crea sesión y redirige a Inicio.
Extensiones	2a) Credenciales inválidas → mostrar error; 2b) 5 fallos → bloquear temporalmente
Reglas de negocio	Contraseña ≥ 8 caracteres, 1 mayúscula, 1 dígito
RF/RNF relacionados	RF-01, RNF-SEC-01

3.3 Diagrama de casos de uso (PlantUML)



3.4 Diagrama de actividad del UC-02 (opcional)



Consejo: Usa casos de uso para acordar *flujos y límites* con el cliente; y usa historias para estimar y planificar sprints.

4) Matriz de trazabilidad (REQ ↔ HU ↔ UC ↔ Test)

Requisito	Historias	Casos de uso	Pruebas
RF-01 Registro	HU-01	UC-01	TC-01 Registro válido; TC-02 Email inválido
RF-02 Ver eventos	HU-02	UC-03	TC-10 Búsqueda por texto
RF-03 Favoritos offline	HU-03	UC-04	TC-20 Modo avión

Mantén la matriz viva: cada cambio de RF debe reflejarse en HU, UC y pruebas.

5) Instrucciones paso a paso para la entrega

1. **Selecciona tu proyecto** y define el **alcance** inicial.
2. Redacta el **SRS (IEEE 830)** siguiendo la plantilla de la sección 1, con **ejemplos concretos**.
3. Escribe al menos **6 RF** y **6 RNF** (seguridad, rendimiento, accesibilidad, disponibilidad).
4. Documenta **3–5 historias** con su tabla y criterios Gherkin.
5. Especifica **2 casos de uso** completos y genera su **diagrama** (PlantUML).
6. Construye la **matriz de trazabilidad**.
7. Pasa una **revisión por pares** (checklist de calidad) y ajusta.

Checklist de calidad:

- ☐ RF/RNF son medibles, no ambiguos y verificables.
- ☐ Historias cumplen INVEST y tienen criterios Gherkin.
- ☐ Casos de uso incluyen flujos alternos y reglas.
- ☐ La trazabilidad cubre todos los RF críticos.
- ☐ Terminología consistente y glosario presente.

6) Entregables (carpeta sugerida)

docs-requerimientos/

─ SRS_IEEE830.md	# Documento principal
─ HISTORIAS_USUARIO.md	# Tabla + Gherkin
─ CASOS_DE_USO.md	# Especificación + PlantUML
─ TRAZABILIDAD.md	# Matriz REQ↔HU↔UC↔Test
└─ ANEXOS.md	# Mockups/DER/notas

Cada archivo debe incluir fecha, versión y responsable.